

Use the strace utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved

Code:

```
GNU nano 8.7.1 sample.txt *
Hello World
This is a sample text file for strace.
```

Output:

```
priyanka@DESKTOP-HOIS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practical/Week 2$ cat sample.txt
Hello World
This is a sample text file for strace.
priyanka@DESKTOP-HOIS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practical/Week 2$ strace --version
Command 'strace' not found, but can be installed with:
sudo apt install strace
priyanka@DESKTOP-HOIS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practical/Week 2$ sudo apt update
sudo apt install strace

= 0
read(3, "# Common strings shared across a...", 4096) = 4096
read(3, "", 32) = 0
close(3) = 0
statx(AT_FDCWD, "/usr/share/coreutils/locales/cat", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFDIR|0755, stx_size=4096, ...}) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/cat/en-US.ftl", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
rt_sigaction(SIGPIPE, {sa_handler=SIG_DFL, sa_mask=[PIPE], sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7d6d33445cb0}, {sa_handler=SIG_IGN, sa_mask=[PIPE], sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7d6d33445cb0}, 8) = 0
rt_sigaction(SIGSEGV, {sa_handler=SIG_DFL, sa_mask=[RTMIN RT_1], sa_flags=SA_RESTORER, sa_restorer=0x7d6d33445cb0}, {sa_handler=0x566ffa5a20c0, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA_SIGINFO, sa_restorer=0x7d6d33445cb0}, 8) = 0
rt_sigaction(SIGBUS, {sa_handler=SIG_DFL, sa_mask=[RTMIN RT_1], sa_flags=SA_RESTORER, sa_restorer=0x7d6d33445cb0}, {sa_handler=0x566ffa5a20c0, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA_SIGINFO, sa_restorer=0x7d6d33445cb0}, 8) = 0
ioctl(1, TCGETS2, {c_iflag=ICRNL|IXON, c_oflag=NUL|CR0|TAB0|BS0|VT0|FF0|OPOST|ONLCR, c_cflag=B38400|CS8|CREAD, c_lflag=ISIG|ICANON|ECHO|ECHOE|ECHON|IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
brk(0x567005dab000) = 0x567005dab000
statx(AT_FDCWD, "sample.txt", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=STATX_BASIC_STATS|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFREG|0777, stx_size=51, ...}) = 0
openat(AT_FDCWD, "sample.txt", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0777, st_size=51, ...}) = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x80, 0), ...}) = 0
splice(3, NULL, 1, NULL, 1048576, 0) = -1 EINVAL (Invalid argument)
pipe([4, 5]) = 0
fcntl(4, F_SETPIPE_SZ, 1048576) = 1048576
splice(3, NULL, 5, NULL, 1048576, 0) = 51
splice(4, NULL, 1, NULL, 51, 0Hello World
This is a sample text file for strace.
) = 51
splice(3, NULL, 5, NULL, 1048576, 0) = 0
close(5) = 0
close(4) = 0
close(3) = 0
sigaltstack({ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=8192}, NULL) = 0
munmap(0x7d6d337f7000, 12288) = 0
exit_group(0) = ?
+++ exited with 0 +++
priyanka@DESKTOP-HOIS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practical/Week 2$ strace cat sample.txt 2> task2.2_trace.txt
Hello World
```

The `strace` command was used to trace the system calls generated by `cat sample.txt`. The `execve()` system call executes the `cat` program. `openat()` opens the file, `read()` reads its contents, `write()` displays the contents on standard output, and `close()` closes the file. These system calls allow the user-level `cat` program to request file and I/O services from the Linux kernel.